University of Hawai■i at M■noa · Campus Energy Project

Campus Grid Load Forecasting

Project Documentation & User Guide

FY2025 · Substation 88162154 · XGBoost Daily Profile Model

DATA SOURCE
15-min meter CSV
35,040 readings

WEATHER
Open-Meteo API
ERA5 reanalysis

MODEL
XGBoost (24 models)
One per hour of day

Table of Contents

1 Setting Up Your Environment

Prerequisites

Before running any code you need three things installed on your computer. Each link below takes you to the official download page.

Tool	Where to Get It	Notes
Python 3.10+	python.org/downloads	Check 'Add to PATH' during install
VS Code	code.visualstudio.com	Free code editor from Microsoft
Python extension	VS Code Extensions panel	Search 'Python' by Microsoft

Your Project Folder

Create a folder called **uh_forecast** anywhere on your computer (Desktop is fine). Place these files inside it:

```
uh_forecast/
■■■ substation_totalized_kw_15_min_fy25.csv
■■■ step1_pipeline.py
■■■ step2_features.py
■■■ step3_train.py
■■■ step4_visualize.py
■■■ step5_forecast.py
```

Create a Virtual Environment

A virtual environment keeps this project's packages isolated from other Python programs on your computer. Open the VS Code terminal (**Ctrl + `**) and run:

```
# Navigate into your project folder first
cd C:\Users\YourName\Desktop\uh_forecast

# Create the virtual environment
python -m venv venv

# Activate it (Windows)
.\venv\Scripts\Activate.ps1
```

If you see a red error about 'running scripts is disabled', run this one-time fix first:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

When the venv is active you will see **(venv)** at the start of your terminal prompt. Then install all packages:

```
pip install pandas numpy matplotlib scikit-learn xgboost \
            openmeteo-requests requests-cache retry-requests scipy
```

> **NOTE**
> You only need to install packages once. After that, just activate the venv each time you open VS Code and you are ready to go.

2 Running the Scripts in VS Code

Run the five scripts in order. Each one saves output files that the next step reads — so the order matters. Never skip a step.

Step	Command	Time	What it produces
1	`python step1_pipeline.py`	~2 min	profiles.csv, daily_features.csv, hourly_merged.csv
2	`python step2_features.py`	~30 sec	X_train.csv, X_test.csv, Y_train.csv, Y_test.csv
3	`python step3_train.py`	~5 min	xgb_models.pkl, predictions_*.csv, model_comparison.csv
4	`python step4_visualize.py`	~30 sec	7 PNG plot files
5	`python step5_forecast.py`	~30 sec	forecast_YYYY-MM-DD.png

Two Ways to Run a Script

■ Play Button

Open any .py file in VS Code. Click the triangle button in the top-right corner. Output appears in the terminal at the bottom. Easiest method.

$ Terminal

Make sure (venv) is visible in your terminal. Type the python command and press Enter. Gives you more control and clearer error messages.

Forecasting a Specific Future Date

After step 3 has trained the models, you can forecast any date within the next 7 days by passing a **--date** argument:

```
# Forecast tomorrow (default)
python step5_forecast.py

# Forecast a specific date
python step5_forecast.py --date 2025-06-15
```

3 What Each Script Does

step1_pipeline.py

Data Ingestion & Reshaping

This is the foundation of the entire project. It does three things in sequence. First it loads your 15-minute substation CSV — the kw column is already instantaneous power demand in kilowatts so no conversion is needed, it just averages the four readings per hour into one hourly value. Second it calls the Open-Meteo API (free, no account needed) and downloads a full year of hourly weather for UH Manoa — temperature, solar radiation, humidity, wind speed, and cloud cover. This only happens once and is cached locally so re-runs are instant. Third it merges load and weather on their shared timestamps and reshapes the data so each row is one day and each column is one hour of that day, producing a 364-day by 24-hour profile matrix.

Output files:

- profiles.csv — the 364 × 24 load profile matrix (your model target)
- daily_features.csv — one row of weather summaries per day
- hourly_merged.csv — full hourly dataset for reference and plotting

step2_features.py

Feature Engineering & Train/Test Split

Takes the outputs from step 1 and builds the full feature set the model trains on. Raw weather alone is not enough — the model also needs to know what load looked like recently. This script adds lag features (yesterday's complete 24-hour load curve, the same weekday last week's curve, and two days ago), a 7-day rolling mean profile capturing the recent typical shape, and rolling weather statistics. It also runs PCA on the training profiles and prints how many components capture how much variance — on your campus data just 3 components explain 97 percent of the shape variation. Finally it performs a temporal train/test split reserving the last 60 days for testing. This is always done by date order, never randomly, so the model only ever learns from the past.

Output files:

- X_train.csv / X_test.csv — feature matrices (131 columns each)
- Y_train.csv / Y_test.csv — target profile matrices (24 columns each)

step3_train.py

Model Training & Evaluation

Trains four models in increasing complexity and evaluates each on the held-out test set so the comparison is fair. The persistence model predicts tomorrow equals yesterday and requires no training — it is the floor that every real model must beat. The mean profile model computes the average load curve for each day of the week. The Ridge regression model fits a linear equation to all 24 hours simultaneously with regularization to prevent overfitting. The XGBoost model trains 24 separate gradient-boosted tree ensembles, one per hour of the day, because midnight load and 2pm load are driven by completely different physics and benefit from learning different feature weights. At the end it prints a comparison table showing MAE, RMSE, MAPE, peak error, peak timing error, daily energy error, and profile correlation for all four models.

Output files:

- xgb_models.pkl — your 24 trained XGBoost models (needed for step 5)
- predictions_xgb.csv / predictions_ridge.csv — test day predictions
- model_comparison.csv — full metrics summary table

step4_visualize.py

Plot Generation

Generates all seven diagnostic and results plots. Nothing new is trained here — it reads the CSV files saved by previous steps and creates charts. The plots cover exploratory data analysis (heatmap, day-of-week profiles, load vs temperature, autocorrelation), model evaluation (predicted vs actual curves for individual test days, per-hour error comparison across models), and error analysis (signed error heatmap by hour and day of week). All plots are saved as PNG files in your project folder and can be opened directly inside VS Code by clicking on them in the file explorer panel.

Output files:

- plot1_heatmap.png through plot7_error_heatmap.png

step5_forecast.py

Live Forecast Generation

Uses your trained XGBoost models to forecast the load profile for any future date up to 7 days ahead. It loads the saved models from step 3, calls the Open-Meteo 7-day forecast API to get projected weather for UH Manoa on your chosen date, builds the feature vector using the forecast weather and your historical profiles as lag features, runs each of the 24 hour-models, and assembles the full predicted curve. It prints a table of predicted kW for each hour with a visual bar chart and saves a PNG showing the forecasted load profile alongside the weather inputs that drove it.

Output files:

- forecast_YYYY-MM-DD.png — predicted load profile with confidence band

4 Understanding Your Output Plots

After running step4_visualize.py you will have seven PNG files in your project folder. Click any one in the VS Code file explorer to preview it. Here is what each one shows and what to look for.

`plot1_heatmap.png`

Full-Year Campus Load Heatmap

WHAT IT SHOWS:

Your entire year of campus load in one image. Every row is one day, every column is one hour, and the color encodes power demand — darker red means higher load, lighter yellow means lower.

LOOK FOR: The daily rhythm appears as a dark horizontal band in the middle of each row. Horizontal light-colored bands across the full row are semester breaks and holidays where campus activity drops significantly. This is your best opening slide for any presentation.

`plot2_dow_profiles.png`

Average Load Profiles by Day of Week

WHAT IT SHOWS:

The average 24-hour load curve broken out by day of the week. Left panel shows Monday through Friday. Right panel shows Saturday and Sunday with the weekday average overlaid as a dotted line.

LOOK FOR: All five weekdays should look very similar — a ramp up around 6–8am, a sustained afternoon peak from AC, then a gradual evening decline. Saturday and Sunday should be roughly 10–15% lower and flatter. This confirms that day-of-week is a meaningful feature for the model.

`plot3_load_vs_temp.png`

Load vs. Temperature Scatter

WHAT IT SHOWS:

Left panel: every dot is one hour of the year. X-axis is temperature, Y-axis is load in kW, dots colored by hour of day. Right panel: the same data grouped into temperature bands as box plots.

LOOK FOR: A clear upward trend confirms air conditioning is the dominant load driver. Dots colored by hour reveal that midday hours cluster at high temperatures and high load simultaneously. This justifies using temperature and solar radiation as the primary weather features.

`plot4_acf.png`

Autocorrelation Function (ACF)

WHAT IT SHOWS:

Shows how correlated today's load is with load from N hours ago, for every lag from 1 to 200 hours. The yellow dashed lines mark the 95% statistical significance threshold.

LOOK FOR: Two large spikes should be clearly visible — at lag 24 (yesterday same hour) and at lag 168 (one week ago same hour). These spikes mathematically prove that lag features belong in your model. If the spikes were absent there would be no point adding them.

`plot5_forecast_days.png`

Predicted vs. Actual — Six Test Days

WHAT IT SHOWS:

Six individual test days shown side by side. Orange solid line is the actual measured load from your meter. Teal dashed line is the XGBoost prediction. The shaded band around the prediction is the uncertainty range.

LOOK FOR: Look for whether the teal line follows the orange line's shape — they should rise and fall together. Check whether the model gets the peak hour right. Days where the prediction is way off often correspond to unusual events such as holidays or unexpected weather the model had not seen before.

`plot6_hourly_mae.png`

Hourly MAE Comparison Across All Models

WHAT IT SHOWS:

Mean absolute error at each individual hour of the day, with a separate colored line for each of the four models. Lower lines mean smaller error and better performance.

LOOK FOR: All models typically struggle most during late morning to early afternoon when solar-driven AC load is most variable. Overnight errors should be small because base load is stable. If the XGBoost line sits below the other three across most hours your model is working well.

`plot7_error_heatmap.png`

Signed Error Heatmap — Hour × Day of Week

WHAT IT SHOWS:

A grid where each row is a day of the week and each column is an hour. Color shows average signed error — blue means the model consistently underestimates load, red means it consistently overestimates.

LOOK FOR: Ideally this should be mostly neutral gray with no strong patterns. Any consistent color patch reveals a systematic blind spot — for example blue on Monday mornings means the model always underestimates Monday morning load. This plot tells you exactly where and when to improve the model.

5 Common Errors & Fixes

`ModuleNotFoundError: No module named 'xgboost'`

CAUSE
A required package is missing from your virtual environment.

FIX
pip install xgboost

Note: Replace 'xgboost' with whatever package name the error mentions.

`ImportError: cannot import name 'filepost' from 'urllib3'`

CAUSE
Package version conflict between requests-cache and urllib3.

FIX
pip install --upgrade urllib3 requests requests-cache

Note: If that fails: pip uninstall urllib3 requests requests-cache -y then reinstall.

`Python was not found`

CAUSE
Python is not installed or not added to PATH.

FIX
Download from python.org/downloads — check 'Add to PATH' during install

Note: Close and fully reopen VS Code after installing Python.

`venv\Scripts\activate: CouldNotAutoLoadModule`

CAUSE
PowerShell script execution is blocked by Windows security policy.

FIX
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Note: Type Y when prompted. Only needs to be run once.

`(venv) disappeared from terminal prompt`

CAUSE
The virtual environment was deactivated (happens when terminal restarts).

FIX
.\venv\Scripts\Activate.ps1

Note: Run this every time you open a new VS Code terminal.

`403 Forbidden from Open-Meteo API`

CAUSE
Network is blocking the weather API (common on university Wi-Fi).

FIX
Try a personal hotspot or different network

Note: Step 1 only needs internet once — after that it uses a local cache.

6 Quick Reference Card

Every Session Checklist

Each time you open VS Code to work on this project, run these two commands before anything else:

```
# 1. Go to your project folder
cd C:\Users\YourName\Desktop\uh_forecast

# 2. Activate the virtual environment
.\venv\Scripts\Activate.ps1

# Confirm (venv) appears in your prompt, then run your step:
python step1_pipeline.py
```

Key Output Files at a Glance

File	Created by	Used by	Contents
`profiles.csv`	step1	step2, step4	364 days × 24 hourly kW values
`daily_features.csv`	step1	step2	Weather + calendar per day
`hourly_merged.csv`	step1	step4	Full hourly data for plotting
`X_train/test.csv`	step2	step3	131 ML features per day
`Y_train/test.csv`	step2	step3	24-hour target profiles
`xgb_models.pkl`	step3	step5	24 trained XGBoost models
`predictions_xgb.csv`	step3	step4	Model predictions on test days
`model_comparison.csv`	step3	—	MAE, RMSE, MAPE for all models
`plot1–7.png`	step4	—	All diagnostic & results charts
`forecast_DATE.png`	step5	—	Future day load profile forecast

Model Performance Benchmark

These are representative results on your FY25 campus data. Your actual numbers will vary slightly depending on weather API data and random seed.

Model	MAE (kW)	MAPE (%)	Profile r	Beats persistence?
Persistence	~600	~5.7	0.72	— (baseline floor)
Mean Profile	~920	~8.8	0.84	No (worse than naive)
Ridge	~420	~3.9	0.93	Yes

XGBoost	~280	~2.6	0.91	Yes (best overall)